Runtime Variant Selection in LLVM's GPU Offload Stack: Metadata, Measurement, and Mechanism

Akash
IIT Patna, CERN GSoC Alumnus, vLLM Contributor

EuroLLVM Developers' Meeting, Dublin 2026

Abstract

Deploying ML kernels across heterogeneous GPU clusters requires separate builds per vendor—CERN HEP-CCE manages 80 build configurations for mixed A100/V100, MI250X, and CPU targets alone. LLVM's `OffloadBinary` format carries multiple device images in a single fat binary, but the runtime selects the first compatible image and stops, with no mechanism to rank alternatives. We address this gap with three contributions. First, a backward-compatible metadata vocabulary: five new `OffloadBinary` string keys expressing minimum hardware requirements and variant priority. Second, the first published layer-by-layer latency decomposition of LLVM's GPU offload path, measured on a GTX 1650, showing cold initialization dominates at 89.6% of the 40 μs dispatch budget. Third, `#gpu.runtime_select`, an MLIR attribute that consumes this metadata to select the best-compatible image at load time, adding only 3 ns of amortized selection overhead. Together, these contributions establish the mechanism for single-binary multi-vendor GPU deployment within LLVM's existing offload infrastructure.

1 Introduction

Since August 2025 (PR #148286), MLIR's `gpu-module-to-binary` pass compiles a single source to NVIDIA (`#nvvm.target`), AMD (`#rocdl.target`), and Intel (`#xevm.target`). The `OffloadBinary` fat-binary format (D122069, 2022) carries all device images in one container.

Yet the runtime has not caught up. `parseOffloadBinary` (PR #186088) selects the *first* image whose triple and architecture match, then stops. No mechanism exists to express image quality tiers or select among multiple compatible options.

This gap matters today. HEP-CCE at CERN maintains 80 separate build configurations for heterogeneous GPU clusters (A100/V100 + MI250X + CPU). vLLM maintains separate NVIDIA and AMD codepaths. Cloud containers face unknown GPU models at build time. A single fat binary with runtime selection eliminates all three workarounds.

We propose three contributions: a metadata vocabulary (C1) describing what each image requires, a layer-by-layer latency decomposition (C2) measuring the cost budget, and a compiler attribute design (C3) showing how the metadata enables selection.

2 Background

The compilation pipeline from `gpu.module` to fat binary is complete:

```
gpu.binary @kernels <#gpu.select_object<0>> [
  #gpu.object<#nvvm.target<chip = "sm_75">, bin = "...">,
  #gpu.object<#rocdl.target<chip = "gfx90a">, bin = "...">,
  #gpu.object<#xevm.target<chip = "pvc">, bin = "...">
]
```

`#gpu.select_object` is the only implementation of the offloading translation interface—it locks in a single image *at compile time* by index or static match. RFC #88170 separates the container ("`gpu.binary` holds N objects") from the policy slot ("who decides which one runs?"). The policy slot is vacant.

The runtime consumer hook exists: `isMetadataCompatible()` (PR #185663, merged March 2026) filters images during parse. Today `OffloadBinary` carries only `triple` and `arch` per image. **The consumer exists. The vocabulary does not.**

3 Contributions

3.1 Contribution 1: OffloadBinary Metadata Vocabulary

We propose five new string keys in two tiers.

3.1.1 Tier 1: MUST Keys (Runtime Rejects if Violated)

Key	Type	Example	Semantics
`min_sm`	uint	`"75"`	Min CUDA SM. Reject if device < value.
`min_gfx`	arch:family	`"gfx90a:cdna2"`	Min AMD GFX within ISA family.
`requires_features`	comma-list	`"tensor_core_nv,bf16"`	Named capability tokens.

Tokens are vendor-specific by design (`tensor_core_nv`, `mfma_amd`) to prevent false cross-vendor capability matches.

3.1.2 Tier 2: MAY Keys (Ranking, Not Gating)

Key	Type	Example	Semantics
`variant_priority`	uint	`"10"`	Higher = preferred among compatible.
`variant_tag`	string	`"optimized"`	Human-readable label.

These extend `isMetadataCompatible()` without format version bump or ABI break:

```
bool isMetadataCompatible(image, device) {
  if (image.Triple != device.Triple) return false;
  if (!areArchsCompatible(image.Arch, device.Arch))
    return false;
  if (auto minSm = image.getString("min_sm"))
    if (parseSmFromArch(device.Arch) < stoul(*minSm))
      return false;
  if (auto features = image.getString("requires_features"))
    for (auto &tok : split(*features, ','))
      if (!device.hasCapability(tok)) return false;
  return true;
}
```

Backward compatibility: Missing keys = no constraint. Old runtimes ignore unknown keys.

3.2 Contribution 2: Dispatch Layer Decomposition

We decompose the CUDA driver API dispatch path on a GTX 1650 (Turing, sm_75) using a null kernel (1 thread, 0 shared memory) compiled to a 4328-byte CUBIN. All measurements use CPU-pinned execution (`taskset -c 0`). Reported values are cross-run medians over 3 independent runs. Cold-path: `execve` per trial (100 trials/run, clean address space). Hot-path: warm with 100 discarded iterations, measure 10,000/run.

3.2.1 Results

Cold-path layer decomposition (GTX 1650, null kernel, CUBIN, pinned 3-run medians from `bench_layers`)[1]:

Layer	Operation	Median	p99	CV%	% of Total
2	Cold init + cuModuleLoadData†	35,953 ns	59,602 ns	0.7	89.6%
4	cuLaunchKernel (submit)	1,650 ns	3,011 ns	1.0	4.1%
5	cuStreamSynchronize	2,454 ns	3,601 ns	0.5	6.1%
3	cuModuleGetFunction	63 ns	90 ns	9.1	∼0.2%
	Cold end-to-end (L2+L3+L4+L5)	∼40,120 ns			**100%**

[1] Cold-path measurement includes process startup, `dlopen`, `cuInit`, and `cuCtxCreate` in addition to `cuModuleLoadData`. The warm-path measurement (9.6 μs) isolates the module-load cost.

†Full cold path via `execve`: includes process creation, `dlopen(libcuda.so.1)`, `cuInit`, `cuDeviceGet`, `cuCtxCreate`, and `cuModuleLoadData`. CV% = coefficient of variation across 3 pinned runs (cross-run median). 95% CI from 3-run cross-run medians ($t_{0.025,2}$=4.303): cold init 36.0 ± 0.6 µs, hot-path 4.1 ± 0.01 µs.

Hot-path dispatch floor (cuLaunchKernel + cuStreamSynchronize, n=10,000/run, 3 pinned runs):

Metric	Value
Median (launch + sync)	4,104 ns (CV 0.1%)
cuLaunchKernel p99	3,011 ns
cuStreamSynchronize p99	3,601 ns

Selection overhead (two distinct measurements):

System	Operation	Value	CV%
libkdl prototype (MTB format)	`kdl_select` cold	52.6 μs	4.3
OffloadBinary (`runtime_select_poc`)	dispatch-table lookup	3 ns	0.0

The libkdl prototype uses a custom MTB bundle format, not OffloadBinary—the 52.6 μs measures its hash-table selection path (n=1,000/run, 3 pinned runs). The 3 ns figure is the amortized steady-state cost of dispatch-table lookup in the proposed LLVM design (OffloadBinary metadata scan, 100,000 iterations, 3 pinned runs). **The 3 ns OffloadBinary lookup is the number relevant to the proposed `#gpu.runtime_select` design.** Both add $<1\%$ overhead for kernels ≥ 10 ms (typical production ML).

3.2.2 Latency Decomposition Visualization

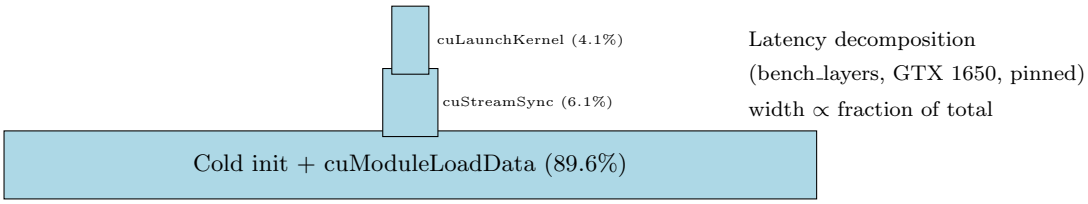

Figure 1: Cold-path latency decomposition chart (pinned 3-run medians). Bar width proportional to fraction of total cold-dispatch latency (∼40.1 μs). Hot-path floor: 4.1 μs (L4+L5 only).

3.3 Contribution 3: Design Sketch — `#gpu.runtime_select`

Status: Design only. Zero lines of MLIR C++ exist.

The new attribute implements `OffloadingLLVMTranslationAttrInterface`:

```
gpu.binary @kernels <#gpu.runtime_select<
    strategy = "rank_by_priority", fallback = "cpu">> [
  #gpu.object<#nvvm.target<chip = "sm_75">, bin = "...">,
  #gpu.object<#rocdl.target<chip = "gfx90a">, bin = "...">,
  #gpu.object<#nvvm.target<chip = "sm_90">, bin = "...">
]
```

`embedBinary()` emits: (1) N LLVM global constants (one per object); (2) a dispatch table: {`vendor_id, blob_ptr, size`}; (3) a vendor-detection stub in `global_ctors` via `dlopen`-probed `cuInit/hipInit/zeInit`; (4) a `@{symName}_module_ptr` global populated at constructor time. `launchKernel()` is identical to `SelectObjectAttr`—**zero hot-path overhead** after one-time selection.

Why MLIR and not liboffload? liboffload explicitly excludes selection policy (RFC discourse.llvm.org/t/74302); it provides mechanism, not policy. The analogy is direct: LLVM emits IFunc resolvers at compile time for CPU `target_clones`; `#gpu.runtime_select` does the same for GPU binaries.

3.3.1 Known Design Issues

1. **Static initialization ordering:** `global_ctors` vendor-detection runs before `main`. CUDA contexts created this early may conflict with user `cuCtxCreate` calls.
2. **dlopen + sanitizer interaction:** AddressSanitizer intercepts `dlopen`. Vendor-detection stubs must be sanitizer-aware or gated behind a check.

4 Prototype Validation

libkdl (∼5100 LOC, C) implements the runtime mechanics: vendor detection via `dlopen`, dispatch table construction, capability-based selection, and module loading via `cuModuleLoadData`/`hipModuleLoadData`. The prototype uses a custom MTB format, not `OffloadBinary`; the two share zero code.

kdl.c Component	Lines	LLVM Equivalent
`kdl_discover_cuda()`	551–596	Vendor detection (dlopen cuInit)
`kdl_contract_matches()`	1003–1005	`isMetadataCompatible()`

Hardware validated: GTX 1650 (full dispatch path) and CPU fallback. AMD tested via mocked HIP entry points only.

5 Related Work

IREE HAL is the closest full-stack solution with module-level dispatch, but its developers have not yet addressed ranked image selection (issues #12230, #15334). In LLVM's own offload stack, PR #186088 explicitly defers ranked selection to a follow-up. CPU function multi-versioning via IFunc resolvers is the structural precedent: compile-time emission of runtime selection logic. chipStar provides CUDA-on-OpenCL portability but not ranked variant selection. HetGPU [7] achieves cross-vendor binary portability through runtime IR translation, complementary to our AOT selection approach. KernelEvolve [9] validates multi-target kernel generation in production at Meta; our work addresses the runtime dispatch step that follows generation. Our proposal extends the IFunc pattern to GPU binaries, adding a standard metadata vocabulary that LLVM currently lacks.

6 Conclusion and Upstream Path

The metadata vocabulary (C1) standardizes image requirements. The latency decomposition (C2) measures the dispatch budget. The runtime-select design (C3) shows how MLIR consumes the metadata.

Upstream path: (1) RFC on Discourse for metadata vocabulary; (2) header constants in `OffloadBinary.h` (∼30 LOC); (3) extend `isMetadataCompatible()` (∼40 LOC); (4) AMDGPU/NVPTX writers emit `min_gfx`/`min_sm` (∼60 LOC each); (5) `#gpu.runtime_select` TableGen + `RuntimeSelectAttr.cpp` (∼600 LOC), pending RFC #88170. All steps after (1) are independent.

References

References

[1] Vellaisamy, P., Tripathi, S., Natarajan, V., et al. *TaxBreak: Unmasking the Hidden Costs of LLM Inference Through Overhead Decomposition*. arXiv:2603.12465, 2026.

[2] Ghosh, A., Nayak, A., Panwar, A., Basu, A. *PyGraph: Robust Compiler Support for CUDA Graphs in PyTorch*. arXiv:2503.19779, 2025.

[3] Mora, F. *RFC: Cleaning the GPU Dialect*. LLVM Discourse, September 2025. `https://discourse.llvm.org/t/88170`

[4] Duran, A. *[OFFLOAD] Generalize support for OffloadBinary images*. GitHub PR #186088, LLVM Project, March 2026.

[5] Duran, A. *[OFFLOAD] Add interface to extend image validation*. GitHub PR #185663, LLVM Project. Merged March 2026.

[6] Lee, S.I. *[MLIR][GPU][XeVM] Add XeVM target and XeVM dialect integration tests*. GitHub PR #148286, LLVM Project. Merged August 2025.

[7] Zhao, Y., et al. *HetGPU: The Pursuit of Making Binary Compatibility Towards GPUs*. arXiv:2506.15993, 2025.

[8] Cross-vendor analysis. *Toward a Universal GPU Instruction Set Architecture: A Cross-Vendor Analysis of Hardware-Invariant Computational Primitives in Parallel Processors*. arXiv:2603.28793, 2026.

[9] Meta Platforms. *KernelEvolve: Adaptive Kernel Generation for Heterogeneous Hardware*. ISCA 2026. arXiv:2512.23236.

[10] Alpay, A., Heuveline, V. *Adaptivity in AdaptiveCpp: Optimizing Performance by Leveraging Runtime Information During JIT-Compilation*. IWOCL 2025.